Unit III: Real-Time Operating Systems (RTOS)

Real Time System:

- A system is said to be real time, if is required to complete its work and deliver its service on time.
- A Real-Time System is a computing system that is required to react to events or inputs within strict, pre-defined time constraints. Correctness depends not only on the logical result of the computation but also on the time at which the result is produced.
- **Timeliness is Key:** Failure to meet the deadline is considered a system failure.

- **For example:** Flight Control System.

Types of Real Time System:

1. Soft Real Time System

- Tasks are performed as fast as possible
- Late Completion of Job is undesirable but not fatal.
- Missing a deadline is undesirable but tolerable and leads to degraded performance (e.g., streaming video, online gaming).
- System performance degrades as more and more jobs are miss deadline.
- Most Operating system are soft real time system.

- **For example:** Multimedia Transmission and reception, cellular Network, Websites, Computer Games, Online Database, ATM

2. Hard Real Time System:

notes.arpanadhikari7.com.np

- There is no flexibility in time constraint It is purely deterministic.
- Missing a deadline is catastrophic (e.g., flight control, medical life support). The worst-case execution time (WCET) must always be guaranteed.

- **For example:** Air traffic control, Vehicle Subsystem control, Nuclear Power Plant Control, Missile launching system, Air bags control in car.

3. Firm Real-Time Systems:

- Deadlines are hard, but if they are missed, the result has no value and is discarded (e.g., network packet routing).
- **Real-Time Clock (RTC):** A crucial component for accurate time stamping and scheduling.

Available on notes.arpanadhikari7.com.np

Components of Real-Time Systems

- Real-time clock
- Real-time operating system
- Tasks and scheduling algorithm
- Inter-task communication
- Interrupt service routines

REAL TIME OPERATING SYSTEM

- **RTOS** is a specialized operating system designed to manage the hardware resources of an embedded system to execute tasks within the required real-time constraints.
- A Real Time operating system is defined as a data processing system in which the time interval required to process and response to input is so small, that it controls the environment. For example automatic car, Street light
- The time taken from input to output task is the response time.
- RTOS responds to input immediately(Real Time).
- Task is completed within a specified time delay.
- In real life situations like controlling traffic signals or nuclear reactor or an aircraft , the operating system has to respond quickly.

Characteristics of RTOS

- **Reliability**
  - Systems must be reliable.
  - Needs to operate without human intervention.
- **Predictability**
  - Should be deterministic(meeting time requirements should be predictable).
- **Performance**
  - Fast enough to fulfill the timing requirement.
- **Compactness**
  - Should be small and efficient to face memory constraints.
- **Scalability**
  - Should be capable of adding/deleting modular components.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Common RTOS Examples

1. FreeRTOS

- Open-source
- highly portable, and focuses on small microcontroller applications.
- Lightweight (suitable for microcontrollers like AVR, ARM Cortex-M)
- Supports tasks, queues, semaphores, mutexes

2. VxWorks

- Commercial RTOS
- Highly reliable with advanced networking and security
- Commercial, widely used in aerospace, defense, and industrial control due to its strong real-time performance and reliability.

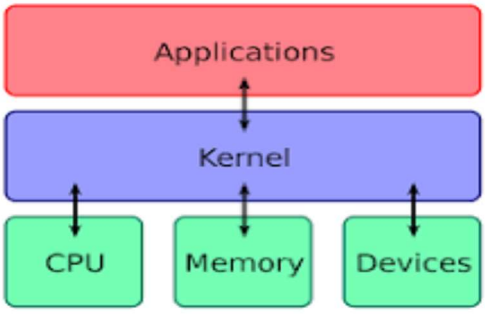
Desktop OS Vs. RTOS:

Desktop OS	RTOS (Real-Time Operating System)
During boot-up, **OS gets control first**, then the application software runs.	During boot-up, **application software gets control first**, then RTOS services start.
Application and OS **share different address spaces**.	Application and OS **share the same address space**.
OS **protects itself carefully** from the user applications.	OS **does not strongly protect itself** from user applications because timing is more important.
General-purpose operating system (used for multiple applications).	Dedicated to a **single embedded or real-time application**.
No fixed timing requirement; services may take variable time.	**Deterministic**; timing for every service is **fixed and predictable**.
High-priority processes may get delayed due to lower-priority tasks (poor priority management).	High-priority tasks **always preempt** low-priority tasks.
Examples: **Windows, macOS, Linux (general purpose)**	Examples: **VxWorks, VxTRX, Nucleus, LynxOS, FreeRTOS**

Available on notes.arpanadhikari7.com.np

Kernel:

- It is the central component of most computer operating system.
- The main aim of kernel is to manage communication between Software and Hardware.
- Software means the user level application.
- Hardware means the CPU, Hard Disk, memory etc.



KERNEL FEATURES:

Process Management:

- The kernel supervise processes, does scheduling, creation, termination and communication.

Interrupt handling:

- Kernel manages system interrupts, prioritizing processes for foreground or background execution.

I/O Device Management:

- Kernel operations through low-level Kernel routes I/O requests from various applications to the suitable I/O devices. The 'device manager' service handles all I/O device system calls known as device drivers.

File System Management:

notes.arpanadhikari7.com.np

- The kernel manages file systems, encompassing tasks like reading, writing, and organizing files on storage devices. It offers a uniform interface for applications to interact with and control files and directories.

Memory Management:

- The kernel supervises memory allocation, deallocation, and protection for processes, including managing virtual memory to provide each process with its own virtual address space.

System Calls:

- The kernel offers system calls as interfaces for user-level applications to request services securely and under control, ensuring a safe interaction between applications and the kernel.

Available on notes.arpanadhikari7.com.np

Error Handling:

- The kernel manages hardware and software errors, preventing system crashes and data corruption through mechanisms like exception handling and error recovery routines.

Interprocess Communication (IPC):

- The kernel enables process communication and data sharing using diverse IPC mechanisms like pipes, sockets, and shared memory.

Security:

- The kernel ensures system security by controlling access, permissions, and isolation among processes and users to prevent unauthorized access and malicious activities.

Scheduling:

- The kernel employs scheduling algorithms to allocate CPU access and duration for processes, striving to achieve balanced resource utilization.

Virtualization:

- Modern kernels often support virtualization, enabling concurrent execution of multiple operating systems or virtual machines on the same physical hardware.

Power Management:

- The kernel optimizes power consumption and extends battery life in portable devices by managing power-related features like sleep modes etc..

REAL TIME KERNEL:

- Real-Time Kernels, also known as Real-Time Operating System (RTOS) kernels, are designed to meet stringent timing requirements and provide predictable behavior for time-critical applications.
- Here are the key features commonly found in real-time kernels:

1. **Deterministic Scheduling**:
   - Real-time kernels use deterministic scheduling algorithms to ensure that tasks with higher priorities are executed before lower-priority tasks.
   - This guarantees that critical tasks get timely access to the CPU, meeting their deadlines.

2. **Preemptive Multitasking**:
   - Real-time kernels support preemptive multitasking, allowing higher priority tasks to interrupt and preempt lower-priority tasks.

Available on notes.arpanadhikari7.com.np

- This feature ensures that time critical tasks can run without delay, even if lower-priority tasks are already executing.

3. **Task Prioritization**:
   - Tasks in a real-time kernel are assigned priorities, and the scheduler uses these priorities to determine the order in which tasks are executed.
   - Higher-priority tasks are given precedence over lower-priority tasks.

4. **Minimal Latency**:
   - Real-time kernels aim to minimize interrupt latency and context-switching time.
   - Low latency is crucial in meeting strict timing requirements and achieving predictable behavior.

5. **Fixed-Time Services**:
   - Some real-time kernels provide fixed-time services for critical operations like context switches, memory allocation, and inter-process communication.
   - These services ensure that these operations complete within predetermined time bounds.

6. **Timers and Clocks**:
   - Real-time kernels often include precise timers and clocks that applications can use to schedule events accurately.
   - These timers are essential for meeting specific timing requirements.

7. **Reliable Task Synchronization**:
   - Real-time kernels ensure effective task cooperation and communication with robust synchronization mechanisms (e.g., semaphores, mutexes, and message queues) to avoid race conditions and deadlocks.

8. **Interrupt Handling**:
   - Real-time kernels efficiently handle hardware interrupts, promptly servicing critical events.

9. **Memory Management**:
   - Real-time kernels offer memory management for tasks, ensuring fast and predictable memory allocation and deallocation.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

10. **Small Footprint**:
    - Real-time kernels are optimized for resource-constrained embedded systems with minimal memory and processing overhead.

11. **Hard and Soft Real-Time Support**:
    - Real-time kernels can cater to both hard real-time and soft real-time needs, where hard real-time systems must meet strict timing deadlines, while soft real-time systems allow some flexibility for occasional deadline misses.

12. **Error Handling**:
    - Real-time kernels incorporate fault handling mechanisms for graceful recovery or safe failure.

TYPES OF REAL TIME KERNEL

1. Monolithic Kernel:

- In this design, the entire set of vital operating system functions and services are consolidated within a single, comprehensive executable file.
- The kernel directly interacts with the hardware, overseeing resource management such as memory, processes, device drivers, file systems, and networking protocols.

Key characteristics of a monolithic kernel:

- **Single Address Space**: All kernel functions and services run in the same address space, allowing for efficient communication and data sharing between different kernel components.

notes.arpanadhikari7.com.np

- **Efficient Performance**: Due to minimal inter-process communication overhead among kernel modules within the same address space, they exhibit enhanced performance.

- **Complexity and Stability**: Monolithic kernels may become intricate and challenging to maintain because of their extensive codebase. A bug in one section of the kernel could impact the overall system stability.

- **Limited Modularity**: Modifying features in a monolithic kernel often demands recompiling the whole kernel, reducing its flexibility and modularity compared to other kernel architectures.

Available on notes.arpanadhikari7.com.np

- **Device Drivers**: Device drivers in a monolithic kernel operate in kernel space, enabling direct hardware access for improved performance but introducing potential security vulnerabilities.

2. Micro Kernel:

- Microkernel design reduces kernel size and complexity by moving most operating system functions to user-space processes.
- Only essential services like inter-process communication and basic memory management remain in the kernel space.
- Other functions, such as device drivers, file systems, and networking protocols, are implemented as separate user-space processes known as "servers" or "services.“

Key characteristics of a microkernel:

- **Modularity**: The microkernel design promotes modularity by keeping the kernel small and focused on essential tasks. Additional services run as separate user-space processes, which can be loaded or unloaded dynamically without the need to restart the entire system.

- **Isolation**: Since most services run in user space, a bug or crash in one service does not affect other parts of the system. This isolation improves system stability and security.

- **Portability**: Microkernel's are often designed to be more portable across different hardware architectures because they rely on a smaller, more standardized kernel interface.

notes.arpanadhikari7.com.np

- **Performance Trade-offs**: While micro kernels aim to improve system stability and security, they can introduce additional overhead due to the need for inter-process communication between user-space services. This overhead can impact overall system performance compared to monolithic kernels.

3. Hybrid Kernel:

- A hybrid kernel blends monolithic and microkernel aspects, aiming to balance performance and modularity.
- It keeps some OS services in kernel space for better performance and implements others as separate user-space processes for enhanced modularity and isolation.

Available on notes.arpanadhikari7.com.np

Key characteristics of a hybrid kernel:

- **Modularity**: Similar to microkernels, hybrid kernels prioritize modularity by running specifi OS services as separate user-space processes. This approach enhances system stability and simplifies the addition or removal of services.

- **Performance Optimization**: To optimize system performance, critical components like device drivers and low-level hardware access are typically retained in kernel space to reduce inter-process communication overhead.

- **Flexibility**: Hybrid kernels offer flexibility by allowing designers to choose which services operate in kernel space and which in user space, enabling customization for specific hardware and performance needs.

- **Isolation**: Through user space separation, a hybrid kernel achieves improved component isolation, minimizing the impact of failures in one part of the system on others.

- **Complexity**: Designing and maintaining hybrid kernels can be more intricate compared to monolithic or microkernels, as they require management of both kernel and userspace components.

4. Exo Kernel:

- The exokernel takes kernel modularity to an extreme level.
- It directly exposes the lowest level of hardware abstractions to applications.
- It allows applications to have explicit control over hardware resource utilization, providing high flexibility and efficiency by minimizing the kernel's role and maximizing application control.

Key characteristics of an exokernel:

- **Resource Management:** The main purpose of an exokernel is to efficiently manage hardware resources with a minimal interface to applications. Rather than offering highlevel abstractions, the exokernel exposes raw hardware resources like CPU time, memory, and network access.

Available on notes.arpanadhikari7.com.np

- **Application Flexibility:** Exokernel empowers applications with greater control over resource management, enabling customization of memory allocation, CPU scheduling, and network communication to meet specific requirements.
- **Modularity:** Exokernel design prioritizes modularity, segregating resource management functions (handled by the exokernel) from higher-level abstractions and services (implemented by library operating systems or "libOSes").

- **Library Operating Systems (libOSes):** In an exokernel setup, conventional OS services such as file systems, process management, and network protocols are user-level libraries (libOSes) built on the exokernel. LibOSes provide higher-level abstractions to applications, while the exokernel concentrates on efficient resource allocation.

- **Performance:** Exokernels strive for high performance in resource-intensive applications by reducing the kernel's involvement and enabling direct hardware access for finegrained resource control.

5. Nano Kernel:

- More minimalistic than a microkernel, providing only essential functionalities for hardware resource management and communication between software components.
- Most OS services, including device drivers, file systems, and networking protocols, are implemented as user-space processes instead of residing in kernel space.

Key characteristics of a nanokernel:

notes.arpanadhikari7.com.np

- **Minimalism**: The nanokernel strives for extreme minimalism, focusing solely on critical functions such as CPU scheduling, memory management, and interprocess communication.

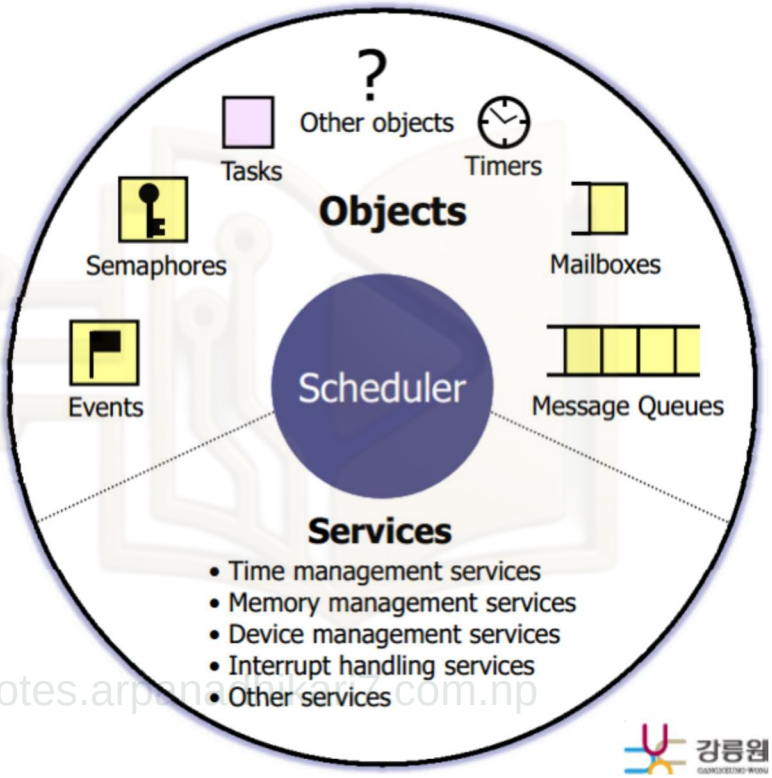
- **Modularity**: Like microkernels, the nanokernel fosters modularity by assigning most services to user-space processes. This enables seamless addition, removal, or replacement of services without impacting the kernel's core.
- **Performance**: The nanokernel's small size and minimal functionality lead to lower overhead and potentially superior performance compared to feature-rich kernel architectures.

Available on notes.arpanadhikari7.com.np

- **Flexibility**: By delegating most services to user-space processes, the nanokernel gains flexibility and adaptability, allowing system customization to meet specific requirements.

- **User-Space Services**: The majority of OS functionality, such as file systems, device drivers, and networking protocols, are implemented as user-space services or servers. They communicate with the nanokernel through well-defined interfaces.

Essential Components of Real Time Kernel:



- ❖ Scheduler
- ❖ Objects
- ❖ Services

SCHEDULER:

- A component of an operating system or RTOS responsible for managing the execution of tasks or processes, deciding which task to run next and allocating system resources efficiently.

Available on notes.arpanadhikari7.com.np

Types of Scheduler:

1. Non-Premptive Scheduling

- **FIFO**(First Come First Served) Scheduling: E.g Ticket Reservation System
- **LIFO**(Last Come and First Served) Scheduling
- **SJF**(Short Job First) Scheduling

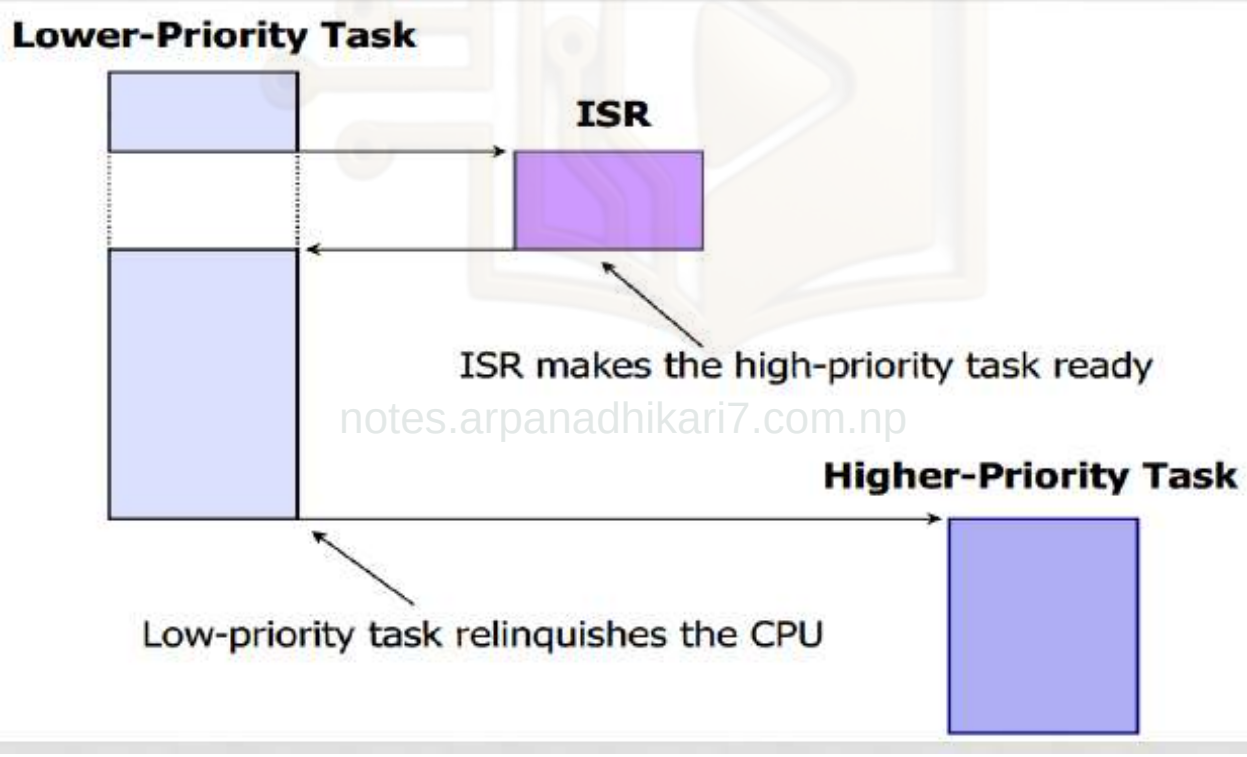
For instance, consider the following tasks and their respective burst times in a real-time system:

Task A: 2 ms
Task B: 5 ms
Task C: 3 ms
Task D: 1 ms

- In SJF scheduling, the task with the shortest burst time will be given the highest priority and executed first. So, in this example, the scheduling order will be:

  Task D (1 ms) Task A (2 ms) Task C (3 ms) Task B (5 ms)

- **Priority Based Scheduling**: Priority is set based on time req. to complete the complete the task.



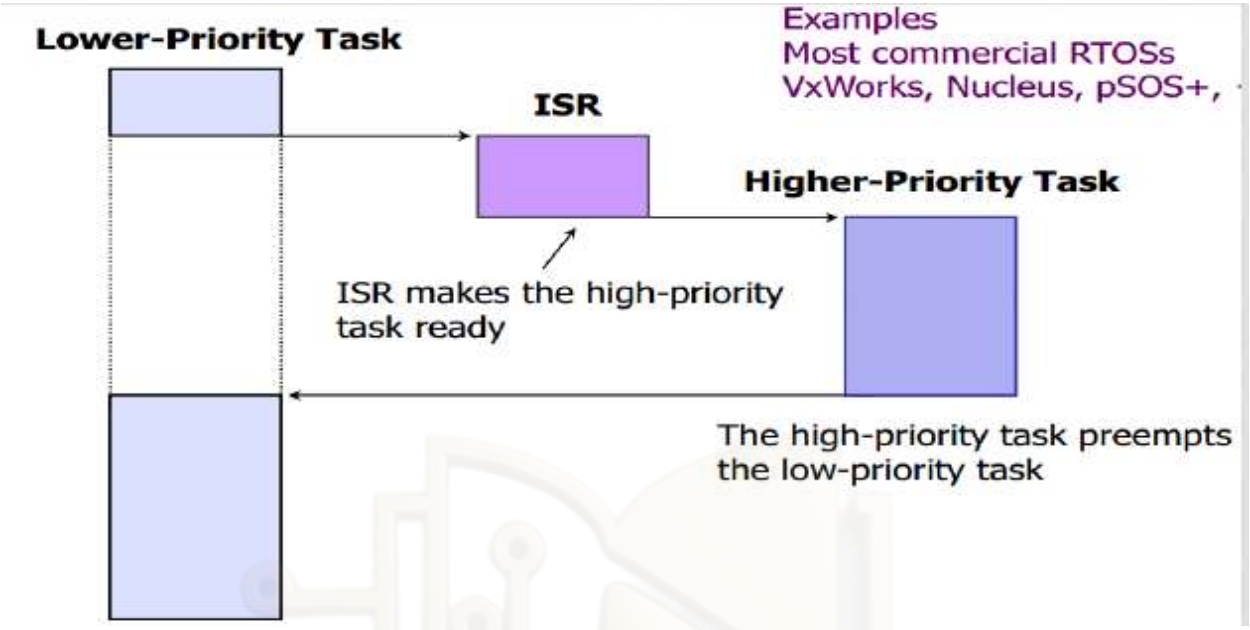
notes.arpanadhikari7.com.np

2. Preemptive Scheduling:

- **SJF(Shortest Remaining Time) Scheduling:** When new process enters the ready queue and checks whether the execution time of new process is shorter than the remaining of the total estimated time for the currently executing process.

Available on notes.arpanadhikari7.com.np

- **Round Robin Scheduling:** Each Process in the ready queue is executed for a pre-defined time slot.
- **Priority Based Scheduling:** It is same as priority based scheduling of non-preemptive except for switching of execution between process.



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Preemptive Scheduling Vs. Non-Preemptive Scheduling:

Preemptive Scheduling	Non-Preemptive Scheduling
The resources are allocated to a process for a **limited time**.	Once resources are allocated to a process, it **holds them until completion** or until it switches to waiting state.
Process **can be interrupted** in between.	Process **cannot be interrupted** until it terminates or goes to waiting state.
If a high-priority process frequently arrives, **low-priority processes may starve**.	If a process with long burst time is running, **short processes may starve**.
Has **overheads** due to frequent context switching.	**No overheads**, since switching occurs only when the current process finishes or waits.
More **flexible** and suitable for real-time systems.	More **rigid** and less suitable for strict real-time behavior.

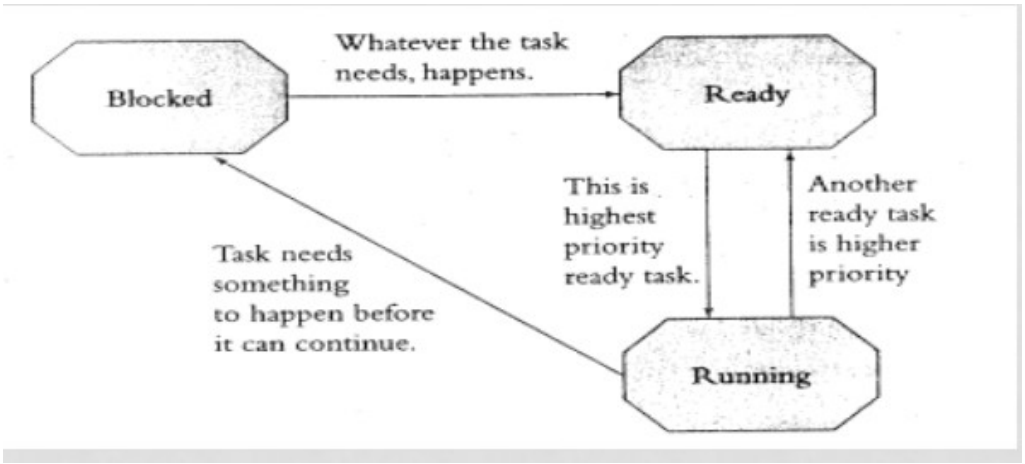
TASK AND ITS STATES:

Task:

- a simple subroutine.
- set of program instructions that are loaded in memory.

Task states:

- Running or Executing, Blocked(waited, dormant, delayed), Ready(pending, suspended).



Available on notes.arpanadhikari7.com.np

Task = Code + Data + State.

- Task State is stored in a TCB(Task Control Block).

TCB:



Task Scheduling

- **Task Scheduling** is the process by which the RTOS kernel determines which task, from the pool of tasks ready to run, should be allocated the CPU next.
- The scheduler's goal in an RTOS is typically **predictability** and **meeting deadlines**, not fairness.

Key Concepts

notes.arpanadhikari7.com.np

- **Task State:** Tasks cycle through states, managed by the scheduler:
  - **Running:** Currently executing on the CPU.
  - **Ready:** Can run, but waiting for the CPU (a higher-priority task is running).
  - **Blocked/Waiting:** Cannot run, waiting for an event (e.g., I/O to complete, time delay, synchronization object).
- **Preemption:** The act of forcibly interrupting a running task to allow a higher-priority task to run. RTOS kernels are typically **preemptive**.

Available on notes.arpanadhikari7.com.np

Context Switching :

- Context switching is the process of **saving the current task's CPU state and loading another task's CPU state**.
- **Context Switching** is the physical mechanism that enables multitasking.
- It is the action of saving the current state (context) of the running task and loading the saved state of the next task to run.
- It occurs when:

    - Scheduler decides to run another task
    - Task becomes blocked
    - Interrupt triggers a higher priority task
    - Time slice expires

The Task Context

The context is the data needed to resume a task exactly where it was interrupted. It primarily includes:

- **CPU Registers:** General-purpose registers, floating-point unit registers, etc.
- **Program Counter (PC):** The address of the next instruction.
- **Stack Pointer (SP):** Pointer to the task's dedicated stack, which holds local variables and function return addresses.
- **Task Control Block (TCB):** A kernel data structure that stores the task's state, priority, and its saved **Stack Pointer**.

Context Switching Process:

notes.arpanadhikari7.com.np

1. **Interrupt:** A software call (e.g., blocking on a resource) or a hardware interrupt (e.g., system clock tick) triggers the scheduler.
2. **Save:** The CPU saves the register values of the **current task** onto its dedicated stack, and the new **SP** value is saved into the task's **TCB**.
3. **Schedule:** The scheduler selects the **next task** to run (the highest priority task in the Ready state).
4. **Restore:** The CPU loads the saved **SP** value from the next task's **TCB**, and then restores (pops) the register values from that task's stack.
5. **Resume:** The new task resumes execution from the instruction pointed to by its restored **PC**.

Available on notes.arpanadhikari7.com.np

Note: Context switch time is **overhead** and must be minimized to maintain real-time performance.

Task Synchronization:

- **Task Synchronization** refers to the techniques used to coordinate the execution of multiple tasks and manage safe access to **shared resources** (like global variables, hardware buffers, or peripherals).
- Synchronization ensures **proper coordination between tasks** when they share resources.

Without synchronization, issues occur:

- Race conditions
- Deadlock
- Data corruption
- Priority inversion

Common Synchronization Problems

- **Race Condition:** Two tasks access shared resource simultaneously → unpredictable results.
- **Deadlock:** Two tasks wait for each other's resources → system freeze.
- **Priority Inversion:** Low-priority task holds resource needed by high-priority task → high priority task is blocked.

- RTOS uses *priority inheritance* to solve it

notes.arpanadhikari7.com.np

Synchronization Mechanisms in RTOS:

- The synchronization mechanisms in an RTOS are primitives designed to **coordinate the execution of tasks** and ensure **exclusive, safe access to shared resources** to prevent data corruption and race conditions.
- Here are the most common mechanisms:

1. Mutex (Mutual Exclusion Semaphore)

A **Mutex** is a lock object specifically used to enforce **mutual exclusion** for a shared resource or a section of code (known as the **Critical Section**).

Available on notes.arpanadhikari7.com.np

- **Function:** Ensures that only one task can access a critical section at any given time.
- **Mechanism:** A task must "take" (acquire) the Mutex before entering the critical section and "give" (release) it upon exit. If a task tries to take an already-held Mutex, it is **blocked** until the owner releases it.
- **Key Feature:** Many RTOS implementations (like FreeRTOS) include a feature called **Priority Inheritance** to mitigate the **Priority Inversion** problem (where a high-priority task is blocked by a low-priority task that holds the Mutex). When a higher-priority task waits for a Mutex, the low-priority task that owns it temporarily has its priority boosted to that of the waiting task.
- **Analogy:** A key to a single-person bathroom. Only the person with the key can enter, and they must return the key when they leave.

---

2. Semaphores (Binary and Counting)

A **Semaphore** acts primarily as a **signaling mechanism** to coordinate tasks or manage multiple resources.

Binary Semaphore:

- **Function:** Used for **signaling** between tasks or between an Interrupt Service Routine (ISR) and a task. It only has two states: **taken** (0) or **given** (1).
- **Mechanism:** One task or ISR can "give" the semaphore to signal that an event has occurred. A waiting task "takes" the semaphore and proceeds. It is often used to notify a task when a hardware event (like a data-ready interrupt) has completed.
- **Analogy:** A signal flag. One person waves the flag to tell another person to start their action.

Counting Semaphore:

- **Function:** Used to manage access to a pool of **multiple, identical resources**.
- **Mechanism:** It maintains a numerical count, initialized to the total number of available resources. When a task requests a resource, the count is **decremented** (taken). When a task releases a resource, the count is **incremented** (given). If the count is zero, the requesting task is blocked.
- **Analogy:** A counter at a rental shop. It tracks how many items (resources) are currently available.

Available on notes.arpanadhikari7.com.np

3. Message Queues (Queues)

Message Queues are the primary mechanism for **Inter-Task Communication (ITC)**, allowing tasks to safely pass data messages to one another.

- **Function:** Provides a **thread-safe, First-In, First-Out (FIFO) buffer** for messages.
- **Mechanism:**
  - A **sender task** writes a message to the queue.
  - A **receiver task** reads a message from the queue.
  - Queues can block the sender if the queue is full and block the receiver if the queue is empty.
- **Benefit:** They **decouple** the tasks, as the sender doesn't have to wait for the receiver, and vice versa. This is crucial for maintaining real-time deadlines. Queues are also safe for sending data from an ISR to a task.
- **Analogy:** A mailbox where you can drop letters (messages). The mail carrier (receiver task) collects them whenever they are ready.

4. Disabling Interrupts

This is the most direct but least flexible synchronization method, typically used for **very short, highly critical sections** of code.

- **Function:** Ensures that the running task cannot be preempted or interrupted by any other event.
- **Mechanism:** The task calls a function to **disable all system interrupts** before entering the critical section and immediately **re-enables them** upon exit.
- **Caution:** Because this method halts all system responsiveness, including the clock tick and all ISRs, it must be used sparingly and only for code that executes in the absolute minimum time possible (e.g., reading/writing a 32-bit shared variable on an 8-bit processor). Prolonged interrupt disabling can cause tasks to miss deadlines.

Resource Sharing:

- Resource Sharing is the core concept where two or more tasks need to access the same physical or logical entity.

Available on notes.arpanadhikari7.com.np

- Shared Resources Include:
    - **Hardware:** I/O peripherals (e.g., UART, SPI bus), shared memory buffers, external flash chips.
    - **Software:** Global variables, linked lists, data structures, file system entries.

Problems in Resource Sharing:

- If multiple tasks access a shared resource **simultaneously**, the following issues arise:

1. **Race Conditions:**

- When access to a shared resource is not properly controlled, a Race Condition can occur.
- This is a flaw where the system's output depends on the sequence or timing of context switches between tasks.
- Example: Both tasks increment the same counter

Solution:

- Resource access must be protected within a **Critical Section** using **synchronization primitives** like a **Mutex** (Mutual Exclusion) or a **Semaphore** to enforce **mutual exclusion**, ensuring only one task can access the resource at a time.

2. Deadlock :

notes.arpanadhikari7.com.np

- A **Deadlock** is a permanent blocking state where a set of tasks are stalled because each task is waiting for a resource held by another task in the set.
- Deadlock occurs when two or more tasks are **permanently blocked**, waiting for resources held by each other.
- None of the tasks can proceed.

Simple Example

- Task A holds Resource X and waits for Resource Y
- Task B holds Resource Y and waits for Resource X
  → No progress → System freeze

Available on notes.arpanadhikari7.com.np

Conditions for Deadlock (Coffman Conditions)

- Deadlock can only occur if all four of these conditions are met simultaneously:

1. **Mutual Exclusion:** Resources involved are non-shareable (e.g., protected by a Mutex).
2. **Hold and Wait:** A task is currently **holding** at least one resource while **waiting** to acquire additional resources held by other tasks.
3. **No Preemption:** A resource cannot be forcibly taken away from the task holding it; it must be voluntarily released.
4. **Circular Wait:** A closed chain of tasks exists, where each task is waiting for a resource held by the next task in the chain.[9]

Deadlock Prevention Techniques:

- **Resource Ordering:** Number resources and force tasks to acquire them in increasing order.
- **Avoid Hold and Wait:** Acquire all required resources before execution.
- **Timeout Mechanisms:** If task cannot get mutex, it stops waiting after timeout:

  FreeRTOS: xSemaphoreTake(mutex, timeout)
- **Deadlock Detection:** RTOS monitors for cycles in resource allocation graph.
- **Priority-Based Preemption:** Higher-priority task may preempt lower ones (but can cause priority inversion).

3. Priority Inversion:

notes.arpanadhikari7.com.np

- Priority Inversion is a critical scheduling anomaly where a **High-Priority Task (HPT)** is indirectly blocked by a **Low-Priority Task (LPT)**, effectively inverting the intended system priority.
- Priority inversion occurs when a **high-priority task is blocked** by a **low-priority task** because the low-priority task is holding a shared resource.

- **Even worse:** A medium-priority task runs normally while high-priority task waits → **inversion**.

Available on notes.arpanadhikari7.com.np

The Scenario:

1. The **Low-Priority Task (LPT)** acquires a Mutex for a shared resource.
2. The **High-Priority Task (HPT)** becomes ready and **preempts** the LPT.
3. The HPT attempts to access the shared resource, finds the Mutex locked by the LPT, and becomes **Blocked**.
4. A **Medium-Priority Task (MPT)** becomes ready. Since the HPT is blocked, the MPT runs and **preempts** the LPT.
5. The MPT executes for a long time, preventing the LPT from running to completion and releasing the Mutex.
6. The HPT remains blocked while the MPT (a lower-priority task) runs, causing the HPT to miss its deadline.

- The order of execution becomes **MPT →LPT →HPT**, (i.e.inverting the priorities)

Solution for Priority Inversion:

- **Priority Inheritance Protocol (PIP):**

The standard RTOS solution is the **Priority Inheritance Protocol (PIP)**:

- When an **HPT** blocks waiting for a Mutex held by an **LPT**, the LPT is **temporarily elevated** to the priority of the waiting HPT.
- This raised priority ensures that the LPT can run immediately and is protected from preemption by the MPT.
- The LPT quickly releases the Mutex, its priority reverts to normal, and the HPT runs immediately.
- Supported by FreeRTOS and VxWorks mutexes.

notes.arpanadhikari7.com.np

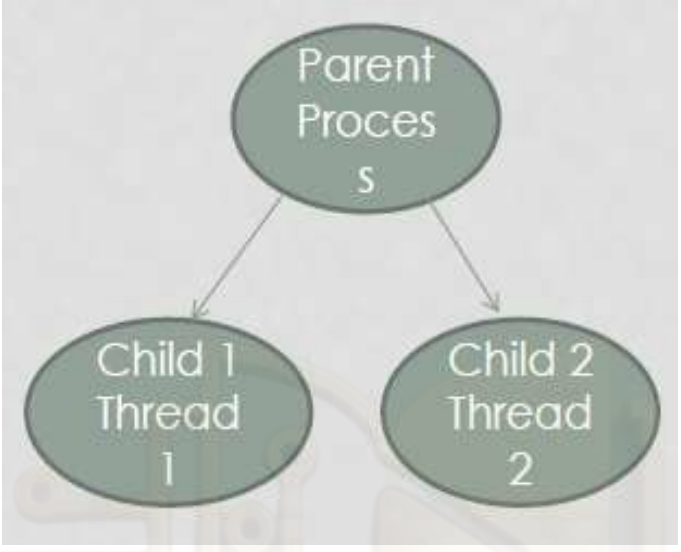
- **Avoid Shared Resources:** Using message queues instead of shared memory prevents many inversion problems.
- **Reduce Critical Section Length:** The shorter the time a task holds a resource, the lower the chance of inversion.
- **Disable Preemption (rarely used):** Used only for very short operations because delays are unpredictable.

PROCESS AND THREAD:

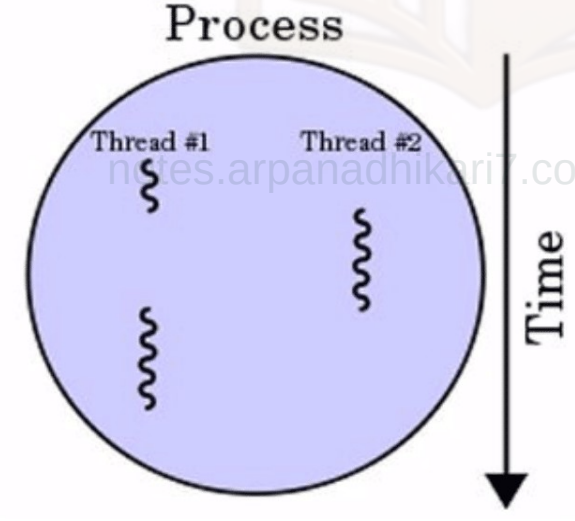
- Each application has multiple process.

Available on notes.arpanadhikari7.com.np

- Each process has its own address space, cpu quota, access to hardware resources and kernel services.
- Process is allocated memory for instruction and data.(process needs memory for code and data.)
- **Example:** Parent process is auto created during system start up. Thread is created dynamically.



Process Vs.Thread:



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Process	Thread
A process is a *program in execution*.	A thread is a *function within an executable* (a lightweight process).
Requires a **separate address space**.	Shares the **same address space** with other threads of the same process.
Inter-process communication (IPC) is expensive and slower.	**Inter-thread communication** is less expensive and much faster.
Context switching between processes is **costly** because it requires saving/restoring full state.	Context switching between threads is **less costly** since threads share resources.
Each process has its **own memory**, program counter, registers, and stack.	All threads share **common memory and resources**, but each thread has its own program counter and stack.
Processes are **heavyweight** and require more system resources.	Threads are **lightweight** and require fewer resources.
Failure of one process **does not affect** other processes.	Failure of one thread can **affect other threads** in the same process.
Created using system calls → **slower creation time**.	Thread creation is **faster** and involves less overhead.

notes.arpanadhikari7.com.np

Multitasking and Multithreading:

- In the context of an RTOS ,the terms Multitasking and Multithreading are often used somewhat interchangeably, as the fundamental unit of execution is typically the **Task** (or **Thread**).
- However, their conceptual origins—the process vs. the thread—define their different levels of abstraction regarding memory and resource sharing.

Available on notes.arpanadhikari7.com.np

1. Multitasking in RTOS (Process/Task-Based):

- In an RTOS, **multitasking** refers to the system's ability to run multiple independent pieces of code (called **tasks**) concurrently.
- Since most embedded systems are resource-constrained and do not use a sophisticated Memory Management Unit (MMU), the "task" in an RTOS often behaves like a "thread" in a general-purpose OS.

Key Characteristics of RTOS Tasks:

- **Unit of Execution:** The basic schedulable unit is the **Task**.
- Isolation: While tasks generally share the same address space (the microcontroller's total memory), they are isolated by having their own dedicated stack area and Task Control Block (TCB).
- **Context Switching:** The RTOS scheduler rapidly performs **Context Switching** between these tasks, giving the illusion that they are running simultaneously.[2]
- **Goal:** To separate system functionality into distinct, manageable units (e.g., one task for motor control, one for communication, one for user interface).

2. Multithreading in RTOS (Shared Memory Concurrency):

- **Multithreading** is the specific concept that all tasks within the system **share the same global memory space**.
- This is the default architecture for almost all embedded RTOS kernel (like FreeRTOS) unless an MMU is present and configured for process isolation.

Key Characteristics:

notes.arpanadhikari7.com.np

- **Shared Resources:** All tasks/threads can directly access **global variables**, **heap memory**, and the **code segment**.
- **Fast Communication:** Tasks communicate easily by reading and writing to shared memory, which is much faster than the Inter-Process Communication (IPC) required in full OS multitasking.
- **Synchronization Necessity:** Because memory is shared, direct access to global data must be protected using **synchronization primitives** (**Mutexes** or **Semaphores**) to prevent **Race Conditions**.[3]
- **Low Overhead:** Switching between threads (tasks) is fast because the memory address space (MMU context) does not need to be swapped, unlike switching between fully isolated processes.

Available on notes.arpanadhikari7.com.np

Multitasking Vs. Multithreading:

Multitasking	Multithreading
Multitasking means executing **multiple tasks (processes)** concurrently.	Multithreading means executing **multiple threads within the same task/process** concurrently.
Each task has its **own memory space**, stack, and context.	Threads **share the same memory space**, code, and resources of the parent task.
Requires **more system resources** (memory, context, stack) than threads.	Requires **fewer system resources** because threads are lightweight.
Context switching between tasks is **slower** and more expensive.	Context switching between threads is **faster** and less expensive.
Tasks are more isolated; failure of one task usually **does not affect others**.	Threads are tightly coupled; failure of one thread can **impact the entire process**.
Better for applications requiring **independent operations**, such as sensor tasks, communication tasks, timer tasks, etc.	Better for applications requiring **parallel operations within the same task**, such as handling multiple events, I/O operations, or computation splits.
Managed by the **RTOS scheduler** at task level.	Managed by the **thread scheduler** inside a task (or RTOS kernel if thread support is provided).
Example: Running a communication task, sensor task, and display task simultaneously.	Example: A communication task having separate threads for sending, receiving, and processing data.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np